第 15 天 • 11/16 • 今日知识点回顾

pivot_table 透视表：index/columns/values/aggfunc 四件套

目标：不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

对着 B 站视频看：视频讲到的每一句，这张 PPT 里都有一条；复习时只翻这份，不用再回去翻分P。

今天结束时，你应该能做到

验收标准（做到这句就算过关）

- 不用 Excel，30 秒内把 1163 行明细做成「城市 × 品类」的实付矩阵，并说出行列各是什么、空白怎么填。

今天的做法（顺序别颠倒）

- 题库地址：file:///D:/DSH工作区/00-学习中心/60-练习环境/Python-Env/data
- 做完记一句：今天卡在哪、错在哪 → 写进 3-每日打卡。

今天看哪几集、读哪一章

这一天的知识：2 集 • 合计约 33 分钟（边看边在本机敲一遍）

P33	Pandas2数据重塑透视	约 18 分钟
P33	【数据的合并和分组聚合】03数据分组聚合	约 15 分钟

教材（已下载到 20-教材与讲义\python-data\，断网也能看）

`https://raw.githubusercontent.com/datawhalechina/joyful-pandas/master/source/Content/ch5.rst`

第五章 变形：pivot / pivot_table / melt / crosstab 全在这一章

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表

- 一句话：index 放行、columns 放列、values 放要算的数、aggfunc 放怎么算。
- 代码：

```
import pandas as pd, pathlib
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
paid = df[df['status'] == 'paid']
pv = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc='sum', fill_value=0)
print(pv.shape)          # (9, 4)
print(pv.round(0).to_string())
```

这一页只讲「pivot_table = groupby + unstack，一步出二维表」。讲完就翻页，下一页是另一个知识点。

知识点 1/7 • pivot_table = groupby + unstack，一步出二维表（续）

★ 本机实测：9 个城市 × 4 个品类，行合计最高的是南京 60,532.00。

• 这个数必须和昨天 groupby(['city','category']).sum().unstack() 的结果完全一致 ——不一致说明有一个用了不同的筛选条件。

知识点 2/7 • aggfunc 可以给多个，也可以给 dict 分别指定

- 一句话：aggfunc=['sum','count'] 出多级列；aggfunc={'amount':'sum','order_id':'count'} 按列分。
- 代码：

```
pv2 = pd.pivot_table(paid, index='city', columns='category',
values='amount', aggfunc=['sum', 'count'], fill_value=0)
print(pv2.columns.tolist()[:4])
pv3 = pd.pivot_table(paid, index='city', columns='category',
values=['amount', 'order_id'],
aggfunc={'amount': 'sum', 'order_id': 'count'}, fill_value=0)
print(pv3.shape)
```

★ 多级列名导出 CSV 后表头是两行，Excel 打开看着乱。给业务看时用

- pv.columns = [f'{a}_{b}' for a, b in pv.columns] 拍平，或者只算一个指标。

知识点 3/7 • margins=True 加一行一列合计

- 一句话：margins=True 加「All」行列，margins_name 改名字。
- 代码：

```
pv4 = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
print(pv4.round(0).to_string())
print('总计：', round(pv4.loc['合计', '合计'], 2))      # 422436.00
```

★ 422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。

- 三次都对上，才说明你对「paid」的定义没变。

知识点 4/7 • fill_value=0 与 dropna 的取舍

- 一句话：没成交的组合到底算 0 还是算「没有」——这是业务问题，不是技术问题。
- 代码：

```
a = pd.pivot_table(paid, index='city', columns='category', values='amount', aggfunc='sum')
print('不填: ', a.isna().sum().sum(), '个空')
b = pd.pivot_table(paid, index='city', columns='category', values='amount',
aggfunc='sum', fill_value=0)
print('填 0: ', b.isna().sum().sum(), '个空')
```

★ 算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。

这一页只讲「fill_value=0 与 dropna 的取舍」。讲完就翻页，下一页是另一个知识点。

知识点 5/7 • crosstab：专做「计数」的透视表，比 pivot_table 短

- 一句话：pd.crosstab(df['city'], df['category']) 直接数每个组合多少单。
- 代码：

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
print(ct.to_string())
ct2 = pd.crosstab(paid['city'], paid['category'], values=paid['amount'],
aggfunc='sum', margins=True, margins_name='合计')
print(ct2.round(0).to_string())
```

★ crosstab 默认 aggfunc='count'（数行数），pivot_table 默认 'mean' ——默认值不同是新手最容易踩的坑：pivot_table 忘了写 aggfunc 会得到平均数。

这一页只讲「crosstab：专做「计数」的透视表，比 pivot_table 短」。讲完就翻页，下一页是另一个知识点。

知识点 6/7 • pivot 和 pivot_table 的区别

- 一句话：pivot 只重塑、不聚合（索引必须唯一）；pivot_table 会聚合（索引可重复）。
- 代码：

```
small = paid.head(3)[['city', 'category', 'amount']]
print(small.pivot(index='city', columns='category', values='amount').to_string())
# 有重复组合时 pivot 报：ValueError: Index contains duplicate entries, cannot reshape
```

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

知识点 7/7 • melt：透视表的逆操作，宽表变长表

- 一句话：pd.melt(pv.reset_index(), id_vars='city') 把列名变成一列「品类」。
- 代码：

```
long = pd.melt(pv.reset_index(), id_vars='city',
var_name='category', value_name='amount')
print(long.shape, long.head(3).to_string(index=False))
```

- （9 城市 × 4 品类 = 36 行长表。做 BI 看板时几乎都要长表，Tableau 尤其吃长表 —— 第 24 天会用到这个转换。）

代码速查：今天这几段，要在本机跑通（1/6）

写 pivot_practice.py，产出四张表并存 CSV：

```
import pandas as pd, pathlib
```

```
DATA = pathlib.Path(r'D:\DSH工作区\00-学习中心\60-练习环境\Python-Env\data')
```

```
df = pd.read_csv(DATA / 'clean_v2.csv', encoding='utf-8-sig')
```

paid = df[df['status'] == 'paid']

```
out = DATA / 'pivot_out'; out.mkdir(exist_ok=True)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（2/6）

1) 城市 × 品类 实付矩阵（9×4）

```
pv = pd.pivot_table(paid, index='city', columns='category', values='amount',
```

```
                    aggfunc='sum', fill_value=0, margins=True, margins_name='合计')
```

```
pv.round(2).to_csv(out / 'city_category_amount.csv', encoding='utf-8-sig')
```

```
print(pv.round(0).to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（3/6）

```
print('总计', round(pv.loc['合计', '合计'], 2))     # 422436.00
```

2) 城市 × 品类 单数矩阵（用 crosstab）

```
ct = pd.crosstab(paid['city'], paid['category'], margins=True, margins_name='合计')
```

```
ct.to_csv(out / 'city_category_count.csv', encoding='utf-8-sig')
```

```
print(ct.to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（4/6）

```
# 3) 城市 × 状态 实付（含 pending/refunded，看退款占比）
```

```
pv3 = pd.pivot_table(df, index='city', columns='status', values='amount',
```

```
                     aggfunc='sum', fill_value=0)
```

```
pv3['退款占比%'] = (pv3['refunded'] / pv3.sum(axis=1) * 100).round(1)
```

```
print(pv3.round(0).to_string())
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（5/6）

4) 宽表转长表

```
long = pd.melt(pv.reset_index().query("city != '合计'"), id_vars='city',
```

```
               var_name='category', value_name='amount')
```

long = long.query("category != '合计'")

```
long.to_csv(out / 'city_category_long.csv', index=False, encoding='utf-8-sig')
```

```
print('长表', long.shape)
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

代码速查：今天这几段，要在本机跑通（6/6）

验收（做到才算过关）：

① 第 1 题 9×4 且总计 = 422,436.00；

② 第 3 题能说出哪个城市的退款占比最高、是多少；

```
③ 能解释 pivot_table 不写 aggfunc 会得到什么（默认 mean），以及为什么危险。
```

先自己在编辑器里写；卡住了再翻教材对答案，别一上来就抄。

最容易错的地方

★ 真实数据几乎一定有重复组合，所以日常用 pivot_table，pivot 只在「确定唯一」的小表上用。报错原文记住：Index contains duplicate entries。

这些句子都是「差一个字、结果差一截」的地方，考前只翻这一页也值。

合上视频，自测一遍（能打勾才算学会）

- [ ] 我能不看资料写出：422,436.00 这个数已经出现三次了（第 12 天 agg、昨天 unstack、今天 margins）。
- [ ] 我能不看资料写出：算「合计」时填 0 和不填结果一样（sum 跳过 nan），但算「平均客单价」时填 0 会把均价拉低 —— 均价场景绝对不要 fill_value=0。
- [ ] 我能不看资料说出：一句话：pivot 只重塑、不聚合（索引必须唯一）；pivot_table 会聚合（索引可重复）。
- [ ] 我能不看题源，把今天用到的每个函数「为什么要有它」说一遍
- [ ] 我能说出今天每个参数/返回值改了会怎样——换个写法结果差在哪
- [ ] 今天这几段代码我在本机都跑通、并看清了输出

这些句子都是「差一个字、结果差一截」的地方；三个勾以上，今天就算过关。

今天收工前，先知道明天学什么

第 16 天 • 时间序列：to_datetime、resample、rolling、同比环比

不用提前学。明天点「① 开始今天的学习」，会换上明天那份 PPT。

收工三件事（5 分钟）

- 今天没跑通的脚本存进 90-我的作业\，命名 2026-11-16-第15天-*.py
- 打卡表「③ 每日打卡」记一笔：今天卡在哪、错在哪（一句话就够）
- 错题明天重做一遍，练习数据在 60-练习环境\Python-Env\data